

Esame Software Engineering (AA 2025/26)

19 Febbraio 2026 - Lab. Colossus - Via salaria 113

Enrico Tronci

*Computer Science Department, Sapienza University of Rome
Via Salaria 113 - 00198 Roma - Italy*

tronci@di.uniroma1.it

<https://raise.uniroma1.it>

Esercizio 4 (20 punti)

Si consideri di nuovo il problema nell'esercizio 3.

Per diminuire il rate di mancate vendite si può pensare di diminuire il tempo di risposta del DB centralizzato. Però questa operazione ha un costo.

Assumendo che $s = l + 1$, il costo totale che tiene conto del rate di mancate vendite R e del tempo di risposta del DB è:

$$J = -a \cdot l + b \cdot R \quad (1)$$

1 Obiettivo

Si vuole scegliere il valore di l in modo da minimizzare il valore atteso del costo J calcolato con M simulazioni Montecarlo. Il calcolo dell'ottimo viene effettuato considerando i valori di l del tipo $l = 0.1 \cdot (1+k)$ con $k = 0, 1, \dots, G$ e selezionando quello che minimizza la funzione obiettivo.

2 Parametri

I parametri della simulazione sono quelli dell'esercizio 3 in cui però i valori di l ed s non sono presenti (poichè vengono calcolati dalla simulazione) ed invece sono letti i seguenti ulteriori parametri.

1. a : parametro funzione obiettivo (valore reale positivo)
2. b : parametro funzione obiettivo (valore reale positivo)
3. G : incremento massimo per l (valore intero positivo)

3 Formato dei parametri di input

Il formato dei parametri di input è lo stesso dell'esercizio 3.

Un esempio di file `parameters.txt` è:

H 234.6
M 100
C 10
A 1.0
B 2.0
Q 3
V 3.0
W 5.0
P 10
S 4
F 2
l 3.1
s 6.3
r 0.01
w 0.01
a 0.8
b 0.2
z 0.1
v 0.2
G 100

4 Formato di output

L'output dell'esercizio è memorizzato nel file `results.txt` la cui prima riga è formattata come indicato nelle istruzioni generali.

Le rimanenti righe del file `results.txt` hanno il formato (R come nell'esercizio 3):

R < rate mancate vendite >
 l < tempo lettura DB >
 s < tempo scrittura DB >
 J < valore funzione di costo >

Un esempio di file `results.txt` è:

2025-01-09-Mario-Rossi-1234567
R 2.3
l 4
s 5
J -56

5 Struttura Sorgenti

Deve esserci almeno una classe, in un proprio file `cpp`, per ognuno dei processi: customer, server, supplier, network, db. Quindi devono esserci almeno i seguenti

files: `customer.cpp`, `server.cpp`, `supplier.cpp`, `network.cpp`, `db.cpp`.